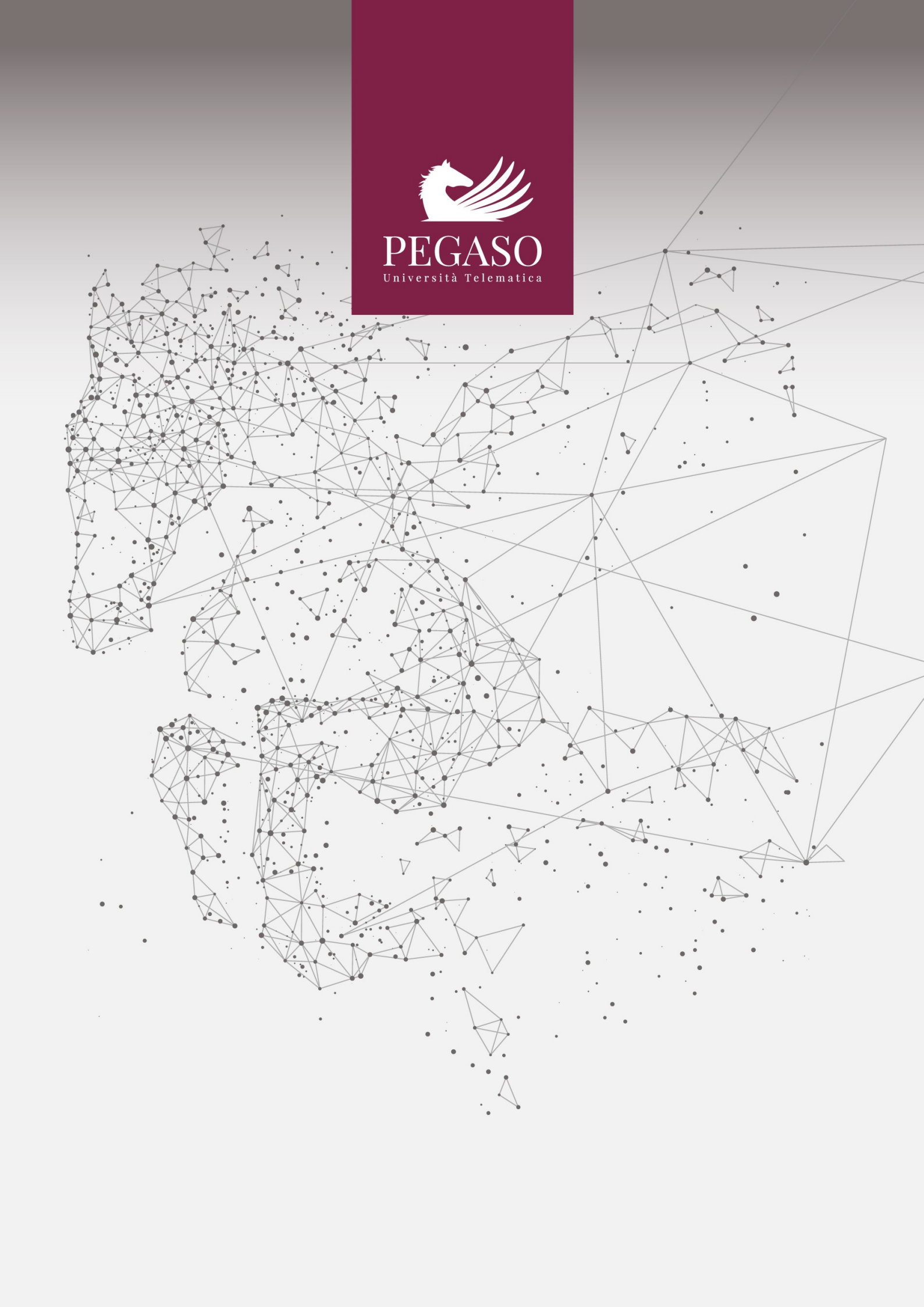

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. INSTALLAZIONE DI PYSPARK....4
3. INTRODUZIONE AL RESILIENT DISTRIBUTED DATASET (RDD)....11
4. PRIME OPERAZIONI CON RDD....13
5. MAPREDUCE IN SPARK....17
BIBLIOGRAFIA....21

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si illustra come installare PySpark e si introduce il Resilient Distributed Dataset (RDD), una collezione di elementi resistente a perdite di dati ed errori e processabile in parallelo su più nodi di un cluster. Si illustrano, quindi, le prime operazioni su RDD, inclusi i metodi per il calcolo di misurazioni e statistiche. Infine, viene presentato l'algoritmo di MapReduce in Spark e vengono illustrate alcune operazioni che l'algoritmo stesso permette di implementare.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Installazione di PySpark

PySpark non è presente nell'installazione base di Anaconda.

Ci sono più possibilità per integrare PySpark in una piattaforma Anaconda. In questa lezione si propongono due metodi alternativi tra di loro. Ne vengono presentati due nel caso in cui il primo metodo non dovesse avere successo.

Il primo metodo prevede di utilizzare il repository di *conda:*

- avviare come Amministratore del file **cmd.exe** (click destro mouse ed «Esegui come Amministratore) solitamente nella cartella C:\Windows\System32
- avviare l'*Anaconda Prompt* digitando il seguente comando:

  C:\ProgramData\Anaconda3\Scripts\activate.bat

  C:\ProgramData\Anaconda3
- digitare il seguente comando:

  conda install –c anaconda pyspark

Il secondo metodo è composto da un numero maggiore di passi. Il primo passo da compiere è verificare se Java 8 è installato.

Per poter controllare la versione di Java:

- menu Start di Windows;
- digitare cmd;
- premere Invio;
- digitare java -version.

Dopo la digitazione precedente, se Java 8 è installato, dovrebbero essere visualizzate alcune stringhe come nell'esempio della Figura successiva.

```
java version "1.8.0_301"
Java(TM) SE Runtime Environment (build 1.8.0_301-b09)
Java HotSpot(TM) 64-Bit Server VM (build 25.301-b09, mixed mode)
```

Se Java 8 non è presente, verrà mostrato un messaggio del tipo: *'java' is not recognized as an internal or external command, operable program or batch file.*

In questo caso, procedere con l'installazione di Java JDK 8 dal sito ufficiale:

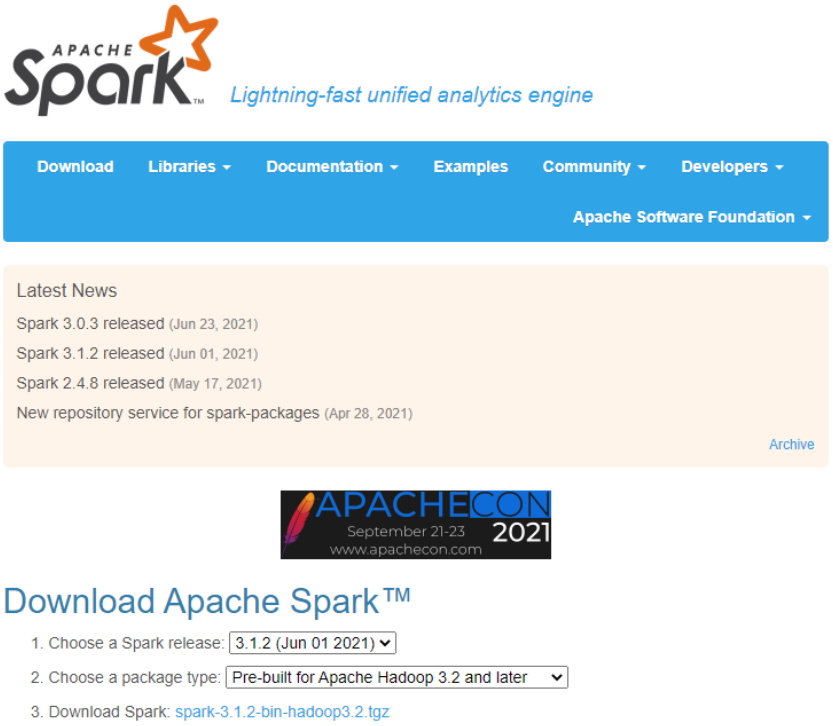
https://www.oracle.com/it/java/technologies/javase/javase-jdk8-downloads.html

Procedere, quindi, installando Apache Spark dal sito:

http://spark.apache.org/downloads.html

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Alla voce «Choose a Spark release» selezionare l'ultima versione stabile disponibile (es. 3.1.2). Alla voce «Choose a package type» selezionare una versione pre-built per l'ultima versione di Hadoop (es. Pre-built for Hadoop 3.2 and later).

Cliccare su link a destra della voce «Download Spark» (es. spark-3.1.2-bin-hadoop3.2.tgz). Dal sito di Apache cliccare su link del mirror site suggerito per il download come indicato nella Figura successiva.



Creare una cartella sul PC per Spark (es. sul desktop o su C:) denominare la cartella «Spark».

È fondamentale che **sia il path che il nome della cartella non contengano spazi.**

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Se fosse stato scelto il Desktop per contenere la cartella Spark, il percorso del file spark-3.1.2-bin-hadoop3.2.tgz dovrebbe avere la seguente forma:

C:\Users\<username>\Desktop\Spark

Copiare, quindi, il file scaricato nella directory "Spark" creata.

Unzip (es. extract here con winzip) del file scaricato (l'operazione di unzip va ripetuta 2 volte da TGZ a TAR e da TAR a cartella).

Se fosse stato scelto il Desktop per contenere la cartella Spark, dopo l'unzip del file, il percorso dovrebbe avere la seguente forma:

C:\Users\<username>\Desktop\Spark\spark-3.1.2-bin-hadoop3.2

Essendo una scelta arbitraria e non univoca, per fare riferimento al percorso e alla cartella appena vista si userà il seguente riferimento **SPARK_HOME.**

Per verificare la corretta esecuzione dei passaggi appena eseguiti:

- nel menu Start di Windows cercare Anaconda prompt;
- andare sul percorso indicato da SPARK_HOME;
- digitare bin\pyspark.

A seguito dell'ultima digitazione, sul prompt di Anaconda verrà visualizzato un output simile a quello riportato nella figura successiva, dove sarà visibile anche la versione di Spark.



Per uscire dal prompt di Anaconda digitare exit() (probabilmente sarà necessario digitare exit() due volte).

Successivamente, creare la cartella "hadoop" nello SPARK_HOME. Creare, inoltre la cartella "bin" dentro la cartella "hadoop". Il percorso di questa nuova cartella "hadoop\bin" verrà identificata come HADOOP_HOME.

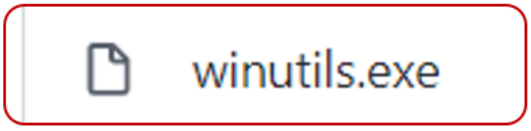
PEGASO
Università Telematica

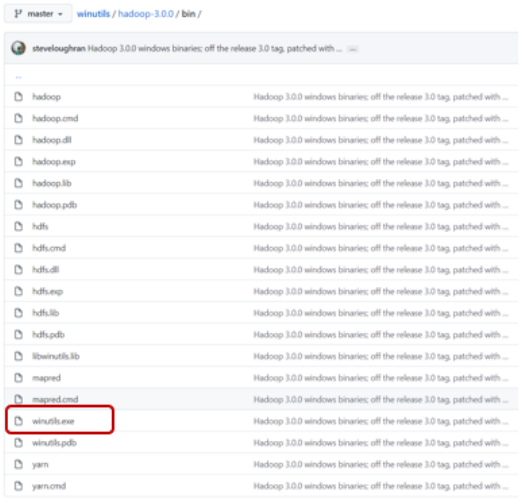
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

HADOOP_HOME sarà dunque uguale a:

SPARK_HOME\hadoop\bin

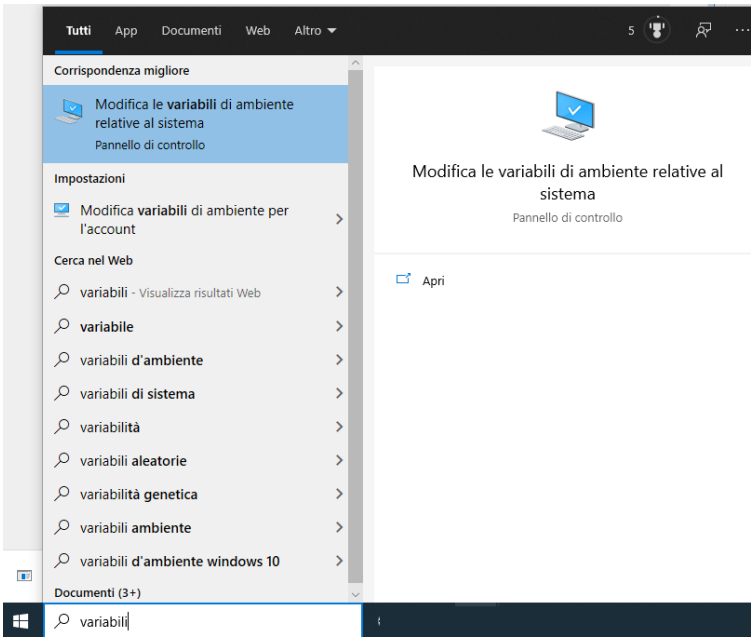
Scaricare la versione .exe (file winutils.exe) per Spark all'indirizzo: https://github.com/steveloughran/winutils/tree/master/hadoop-3.0.0/bin (come mostrato nella figura seguente in cui è stata scelta l'ultima versione di Hadoop, la 3.0.0) e salvare il file in HADOOP_HOME.





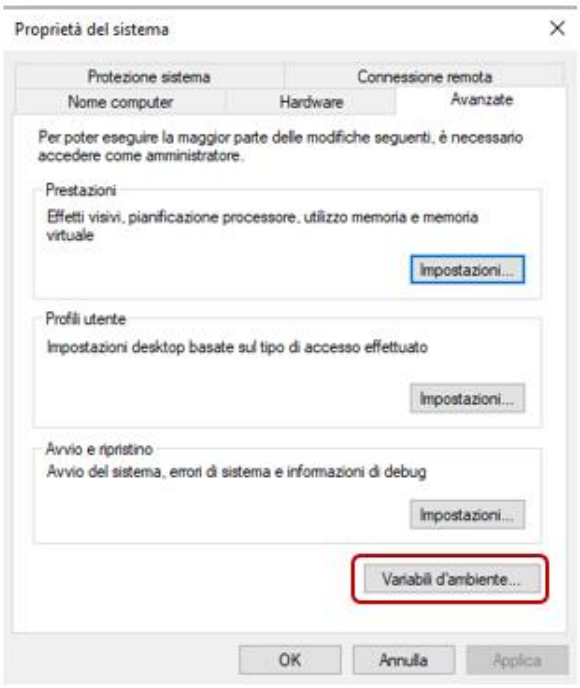
Creare le variabili di ambiente SPARK_HOME e HADOOP_HOME nel seguente modo (si ricorda che è necessario aver effettuato l'accesso al PC come amministratore):

1. Cercare "variabili" nel menu di ricerca dello Start di Windows come mostrato nella figura seguente. Cliccare su "Modifica le variabili di ambiente relative al sistema".



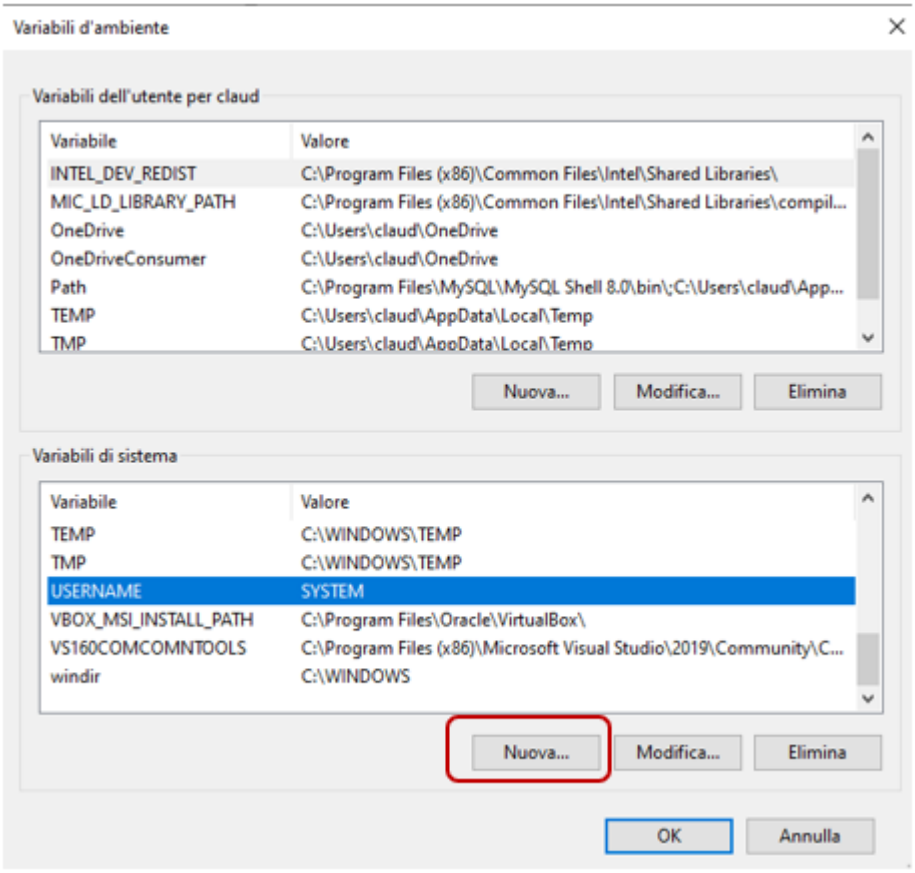
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Cliccare su «Variabili d'ambiente» come mostrato nella Figura seguente.

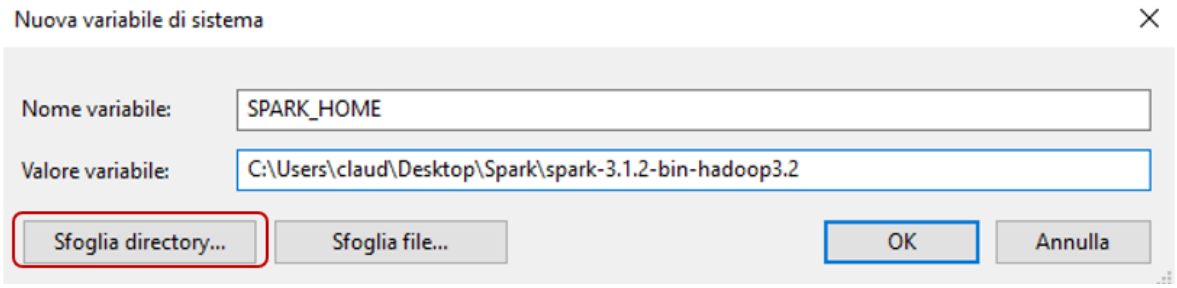


3. Cliccare su «Nuova» in variabili di sistema come indicato nella figura seguente.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

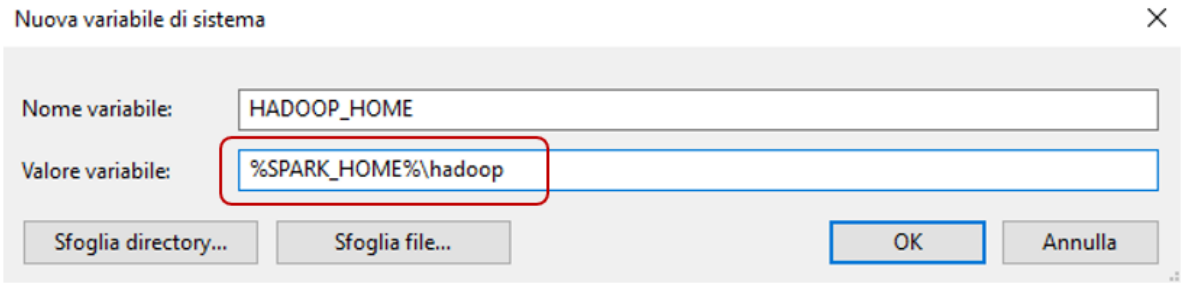


4. Digitare in “nome variabile” la stringa “SPARK_HOME”. “Sfoglia directory” per trovare la cartella di spark. Premere OK. Nella Figura seguente si riporta l’esempio delle operazioni.



5. Ripetere lo stesso procedimento per la variabile d’ambiente HADOOP_HOME. Essendo Hadoop dentro SPARK_HOME, è possibile rendere HANDOOP_HOME relativo a SPARK_HOME scrivendo:

%SPARK_HOME%\hadoop



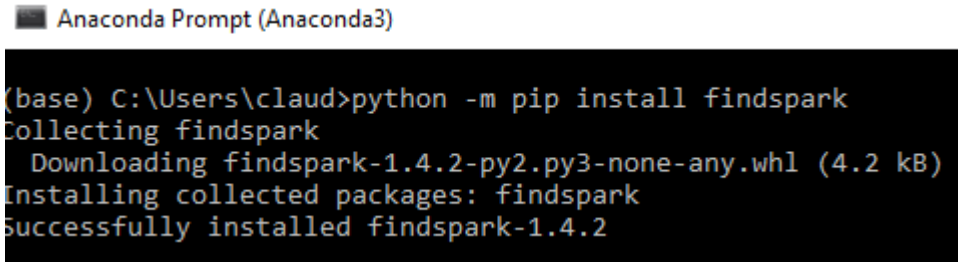
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

6. Aprire nuovamente l'Anaconda Prompt. Digitare:

   python –m pip install findspark

   Questo passaggio è necessario per eseguire Spark da Jupyter Notebook.



7. Di fondamentale importanza è il **riavvio del PC per rendere effettive le variabili di ambiente**.
8. Aprire Anaconda Navigator e «lanciare» Jupyter Notebook. Eseguire il primo test con il seguente codice:

```
import findspark

findspark.init()
print("Terminato senza errori")
```

9. Eseguire il secondo test con il seguente codice:

```
import pyspark

from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()

df = spark.sql("select 'spark' as hello ")

df.show()
```

Per verificare la corretta esecuzione del codice, attendere il termine dell'elaborazione e verificare che l'output sia uguale a quello mostrato nella Figura seguente.

```
+-----+
|hello|
+-----+
|spark|
+-----+
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Introduzione al Resilient Distributed Dataset (RDD)

La prima struttura dati in ambiente PySpark che analizzeremo è il **Resilient Distributed Dataset (RDD).** Per testare tale struttura dati non serviranno realmente dei Big Data (cioè grandissime quantità di dati). Le funzioni che saranno analizzate, infatti, sono applicabili a piccoli dataset come a grandi archivi allocati su cluster. Non è rilevante neanche possedere un cluster Spark in quanto tutto il necessario alla sperimentazione e allo studio sarà incluso in PySpark.

RDD è una **collezione** di elementi, **resistente a perdite di dati ed errori** (da qui Resilient) e processabile in **parallelo.** Ed è proprio la processabilità in parallelo di fondamentale importanza per il contesto Big Data poiché l'elaborazione può essere distribuita su più nodi del *cluster*. Esistono due modi per ottenere un RDD:

1. **parallelizzare strutture dati Python**. Questa è la via più immediata ed è il modo in cui possiamo rimodulare dati già presenti nei nostri programmi in una modalità che sia utilizzabile su un cluster Spark;
2. **caricare dataset memorizzati su file**. Questi possono essere presenti in locale o custoditi su HDFS, il file system di Hadoop, o in altri contesti di memorizzazione.

Due sono anche le **tipologie di operazioni** che possono essere svolte sugli RDD:

1. **trasformazioni** - sottopongono ogni elemento di un RDD a una funzione producendo da questo un altro RDD;
2. **azioni** - eseguono elaborazioni su un RDD generando un risultato che sarà restituito al programma che le ha attivate.

È importante conoscere le tipologie di operazioni possibili perché tale distinzione verte sulla **modalità** e sulle **tempistiche** con cui Spark le esegue. Le *trasformazioni* vengono condotte con un comportamento definito lazy («pigro»). Esse, infatti, non vengono eseguite all'istante, ma solo quando Spark lo reputa assolutamente necessario. Finché si richiederanno *trasformazioni* sui dati, Spark ne prenderà nota per, poi, eseguirle al momento in cui un'*azione* vorrà estrarre un risultato da esse.

Questa strategia è ideale per ottimizzare le elaborazioni. Infatti, tutto ciò che può essere rimandato viene rimandato fino al momento in cui il programma richiede un risultato. Per esempio, con gli RDD è possibile implementare l'algoritmo di *MapReduce*. Esistono metodi per l'operazione di *map* e altri per quella di *reduce*, ma i primi saranno delle *trasformazioni*, i secondi saranno delle *azioni*. Infatti, il *map* è un processo transitorio di predisposizione dei dati, mentre il *reduce* è incaricato di estrapolare realmente un risultato.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Dal punto di vista pratico, sia le *trasformazioni* sia le *azioni* sono espresse da **metodi** da invocare su un **oggetto RDD**. A seconda della loro finalità, riceveranno degli argomenti alcuni dei quali saranno **funzioni Python** o **espressioni lambda**.

Per esempio, tutti i metodi che implementano un *map* riceveranno una funzione o una lambda che indicherà quale trasformazione vogliamo eseguire sugli elementi del RDD.

Si riportano, ora, alcuni **metodi** corrispondenti alle *trasformazioni* più comuni in PySpark:

- **map** – elabora ogni elemento di un RDD sottoponendolo alla funzione passata come argomento;
- **filter** – filtra gli elementi di un RDD utilizzando come criterio la funzione passata come argomento;
- **flatMap** – simile a map con la differenza che per ogni elemento può essere restituita una sequenza di risultati;
- **groupByKey** – raggruppa gli elementi forniti sotto forma di coppie chiave/valore (per ogni chiave, restituisce una sequenza iterabile dei valori corrispondenti);
- **groupBy** – esegue un raggruppamento usando come criterio la funzione passata come argomento;
- **reduceByKey** – esegue un'operazione di reduce raccogliendo gli elementi per chiave;
- **sortByKey** – ordine delle coppie chiave/valore in base alle chiavi;
- **coalesce** – riduce il numero di partizioni prodotte.

Si riportano, inoltre, alcuni **metodi** corrispondenti alle *azioni* più comuni in PySpark:

- **reduce** – aggrega i valori forniti in base a una funzione passata come argomento;
- **collect** – restituisce gli elementi del RDD sotto forma di struttura dati Python;
- **count** – conta gli elementi presenti nel RDD;
- **first** – restituisce il primo elemento del RDD;
- **take** – restituisci i primi elementi di un RDD in quantità pari al numero passato come argomento;
- **saveAsTextFile** – salva un RDD come file di testo;
- **foreach** – esegue una funzione su ogni elemento di un RDD (si può considerare un'alternativa ai tradizionali cicli).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Prime operazioni con RDD

A seguito dell'integrazione di PySpark in Anaconda è possibile utilizzare PySpark nelle celle del notebook Jupyter. Il tempo di risposta sarà, tuttavia, vincolato al **tipo di operazione** che stiamo svolgendo e alla **quantità di dati** coinvolti.

Uno script PySpark consiste in un file Python in cui verranno caricate le librerie di PySpark necessarie e verrà istanziato uno **SparkContext.** Gli **script PySpark** nel *notebook Jupyter* devono avere sempre le seguenti righe iniziali:

```
import findspark

findspark.init()
```

La tipica forma di uno **script PySpark** è la seguente:

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

#codice per l'analisi dei dati

sc.stop()
```

Gli argomenti passati a *SparkContext* sono due:

- il valore '*local*' indica che sarà un cluster locale avviato da PySpark stesso. Ciò è fondamentale poiché, una volta installato il nostro ambiente, non sarà necessario preoccuparsi di creare o meno un cluster per le sperimentazioni;
- la stringa "*Esempio*" è il nome dell'applicazione Spark.

Al termine dello script **si invoca sempre sc.stop()** per indicare che non è più necessario lo *SparkContext*. Questo favorisce un'eventuale liberazione delle risorse da parte di Spark.

Per iniziare a conoscere gli RDD proviamo a caricare dati da una lista Python. Verrà utilizzato il metodo **parallelize()** che accetta come argomento, tra gli altri, proprio una **lista Python**.

Il risultato è la migrazione della lista in un **RDD** per essere analizzata in un cluster Spark come visibile nel codice seguente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

rdd_numeri=sc.parallelize([21,44,45,55])
print(rdd_numeri)

sc.stop()
```

```
ParallelCollectionRDD[0] at readRDDFromFile at PythonRDD.scala:274
```

Sull'oggetto *rdd_numeri* sarà possibile richiamare i metodi per eseguire elaborazioni sui dati contenuti come il conteggio, il minimo, il massimo e la media come riportato nel codice seguente.

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

rdd_numeri=sc.parallelize([21,95,45,34,57,25,88,6,9,18,25,34])
print("Conteggio:",rdd_numeri.count())
print("Minimo:",rdd_numeri.min())
print("Massimo:",rdd_numeri.max())
print("Media:",rdd_numeri.mean())

sc.stop()
```

```
Conteggio: 12
Minimo: 6
Massimo: 95
Media: 38.08333333333333
```

Altri metodi che possono essere invocati sono la somma, la deviazione standard, la varianza o le statistiche come riportato nel codice seguente.

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

rdd_numeri = sc.parallelize([21,95,45,34,57,25,88,6,9,18,25,34])
print("Somma:",rdd_numeri.sum())
print("Deviazione standard:",rdd_numeri.stdev())
print("Varianza:",rdd_numeri.variance())
print("Statistiche:",rdd_numeri.stats())

sc.stop()
```

```
Somma: 457
Deviazione standard: 27.511992334656938
Varianza: 756.9097222222222
Statistiche: (count: 12, mean: 38.08333333333333, stdev: 27.511992334656938, max: 95.0, min: 6.0)
```

Per estrarre singole misurazioni a partire dalle **statistiche** (conteggio, media, deviazione standard, massimo e minimo) una volta calcolate, è possibile calcolarle nel seguente modo:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

rdd_numeri = sc.parallelize([21,95,45,34,57,25,88,6,9,18,25,34])
rdd_stats = rdd_numeri.stats()
print("Media:",rdd_stats.mean())

sc.stop()
```

```
Media: 38.08333333333333
```

Si richiama, quindi, il metodo desiderato direttamente sull'oggetto *rdd_stats*.

In Spark, è sempre importante notare quando ciò che si ottiene **non è più una struttura dati per Big Data**, ma viene tradotto in una normale struttura dati o in un tipo di dato di Python. Tutte le informazioni numeriche ottenute (come i risultati di somma, media, minimo, massimo, ecc.) sono di tipo ***float*** o ***int*** a seconda che abbiano o meno la virgola. **stats**(), invece, restituisce un oggetto **StatCounter** offerto dal package *pyspark.statcounter.*

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

rdd_numeri = sc.parallelize([21,95,45,34,57,25,88,6,9,18,25,34])
rdd_stats = rdd_numeri.stats()
print(type(rdd_stats))

sc.stop()
```

```
<class 'pyspark.statcounter.StatCounter'>
```

Con il metodo *parallelize*() abbiamo visto come trasformare una struttura dati Python in un RDD parallelizzabile su *cluster Spark*. L'operazione inversa, cioè richiedere il risultato di un'operazione estraendolo dall'ambito Spark per riportarlo in una normale struttura dati Python, è possibile con il metodo **collect**().

Nell'esempio che segue si vuole riportare il contenuto di *rdd_voti* in una lista Python (è possibile osservare che il metodo collect() restituisce una lista).

```
print(rdd_voti.collect())
```

```
[6, 8, 8, 9, 5, 6, 6, 8]
```

```
print(type(rdd_voti.collect()))
```

```
<class 'list'>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato del metodo **collect**() può pertanto essere utilizzato con operazioni e metodi tipici delle liste quali *indexing, slicing* e funzioni *built-in.* Nel codice seguente, si riportano alcuni esempi sulla lista ottenuta tramite il metodo collect().

```
lista_voti = rdd_voti.collect()
print("Indexing - lista_voti[2] =",lista_voti[2])
print("Slicing - lista_voti[2:5] =",lista_voti[2:5])
print("Pop - lista_voti.pop() =",lista_voti.pop())
```

```
Indexing - lista_voti[2] = 8
Slicing - lista_voti[2:5] = [8, 9, 5]
Pop - lista_voti.pop() = 8
```

Ad ogni modo non bisogna farsi ingannare dal numero esiguo di dati impiegati. In reali contesti big data, infatti, il metodo *collect*() viene usato **al termine delle operazioni** esclusivamente al fine di ottenere un risultato da esportare o visualizzare. Tutto ciò che richiede la potenza di un cluster per il calcolo parallelo deve comunque passare per gli RDD Spark.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. MapReduce in Spark

Nonostante esistano altre modalità di elaborazione più specifiche in Spark, è importante vedere come gestire **l'algoritmo di MapReduce.** Il MapReduce in Spark ha un'applicazione più elastica rispetto a quella di Hadoop. Ovviamente, anche in Spark ci sono le due funzionalità *map* e *reduce* che hanno i seguenti scopi:

- il *map* opera una **trasformazione** tra i dati che vengono immessi in ingresso, provenienti da un RDD, è un risultato da offrire in output;
- il *reduce* implica un lavoro di **sintesi** sui risultati del *map* finalizzato a estrarne un risultato.

A seconda delle necessità, si possono eseguire più concatenazioni tra queste funzioni. Per esempio, applicando due *map* di seguito per distribuire le operazioni o creare task *map-only* che non usano, quindi, un *reduce*.

L'**argomento** di un *map* consiste in una **funzione**. Questa sarà applicata ad ogni valore del RDD di partenza e i risultati saranno a loro volta incapsulati in un nuovo RDD.

Supponiamo di avere in **una lista Python** un elenco di candidati che hanno sostenuto una prova in un test ricevendo un voto in trentesimi. Il formato della stringa vede all'inizio il cognome, seguito dall'iniziale del nome ed, infine, il numero che rappresenta il voto del test come segue:

cognome n. 28

Semplificando, si assume che tutte le stringhe rispettino questo formato e che le ultime due cifre compongano correttamente un numero intero. Si riporta, di seguito, una lista con le stringhe relative ai candidati formattate come appena descritto.

```
candidati=['Rossi M. 25','Verdi V. 30', 'Marroni B. 19',
           'Gialli E. 22', 'Marroni C. 28', 'Verdi D. 24']
```

Se volessimo estrarre dalle stringhe tutti i voti per poterne conoscere il più basso, dovremmo utilizzare il metodo *map()* chiedendo di eseguire su ogni elemento la seguente **espressione lambda**:

lambda s: int (s[-2:])

che estrae da ogni elemento gli ultimi due caratteri come riportato nel codice seguente.

```
rdd_candidati = sc.parallelize(candidati)
minimo = rdd_candidati.map(lambda s: int (s[-2:])).min()
print(minimo)
```

L'oggetto s rappresenta proprio l'elemento che da RDD viene estratto di volta in volta. Si crea prima *rdd_candidati* per poi eseguire su questo il metodo ***map().*** Il risultato di *map* è anch'esso di tipo RDD, pertanto è possibile richiamare il metodo *min*(). L'esito riporta che il voto più basso conseguito è stato 19.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il codice completo dell'esercizio è il seguente:

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

candidati=['Rossi M. 25','Verdi V. 30', 'Marroni B. 19',
           'Gialli E. 22', 'Marroni C. 28', 'Verdi D. 24']

rdd_candidati = sc.parallelize(candidati)
minimo = rdd_candidati.map(lambda s: int (s[-2:])).min()
print(minimo)

sc.stop()
```

```
19
```

Si applica ora un procedimento leggermente più articolato. Partendo nuovamente da *rdd_candidati* ricaviamo da un *map* un risultato in formato **chiave/valore** dove:

- la **chiave** è formata da cognome e iniziale del nome;
- il **valore** è il voto conseguito.

Al termine otterremo l'elenco di candidati in ordine decrescente in base al voto finale.

Per quanto riguarda **l'ordinamento**, è possibile utilizzare il metodo **sortBy**() che riceve come argomento una funzione o espressione lambda che **evidenzia il criterio per l'ordinamento.**

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

candidati=['Rossi M. 25','Verdi V. 30', 'Marroni B. 19',
           'Gialli E. 22', 'Marroni C. 28', 'Verdi D. 24']

rdd_candidati = sc.parallelize(candidati)
token = rdd_candidati.map(lambda x: x.split())
key_value = token.map(lambda parti: (f'{parti[0]} {parti[1]}',parti[2]))
sorted_key_value = key_value.sortBy(lambda x: x[1], ascending=False)
ris = sorted_key_value.collect()

print(ris)

sc.stop()
```

```
[('Verdi V.', '30'), ('Marroni C.', '28'), ('Rossi M.', '25'), ('Verdi D.', '2
4'), ('Gialli E.', '22'), ('Marroni B.', '19')]
```

Il risultato è una lista Python ottenuta attraverso il metodo **collect**() invocato al termine dello script.

Approfondendo le varie porzioni dello script precedenti è possibile osservare che:

- il primo *map()* lavora sui singoli elementi di *rdd_candidati* separandoli in base agli spazi;

map(lambda x: x.split())

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- il secondo *map*() lavora sul risultato del *map* precedente recuperando, in un **oggetto** di nome ***parti***, le singole porzioni che compongono la stringa. Ad ogni iterazione produce una tupla composta da una concatenazione di cognome e nome e, come secondo elemento, dal voto. Proprio la **tupla** è la struttura dati con cui si compone la **coppia chiave/valore**;

  map(lambda parti: (f'{parti[0]} {parti[1]}',parti[2]))

- il **sortBy**() riceve ogni tupla prodotta dal secondo *map*. Il sortBy dovrà ordinare l'insieme di elementi **in base al secondo valore della tupla**, il voto. Tale criterio di ordinamento viene comunicato estraendo proprio il voto con l'espressione lambda *lambda x: x[1]*. L'argomento *ascending* impostato a *False* serve richiedere un ordinamento decrescente;

  sortBy(lambda x: x[1], ascending = False)

- al termine **collect**() estrae il risultato in una struttura dati.

Come è stato osservato nell'esempio, il voto più alto è stato un 30 conseguito da Verdi V. Finora non è stata utilizzata l'operazione di **reduce** esplicitamente sebbene la funzione **min**() abbia ricoperto proprio tale ruolo. Un *reduce* segue un *map* e permette di **sintetizzare operazioni** nel momento in cui non ci sono delle funzioni apposite.

Si propone ora, un esempio numerico per presentare il MapReduce. Dato un elenco di numeri si vuole calcolare la somma dei pari e dei dispari separatamente. Il codice che permette di ottenere tale risultato è riportato di seguito.

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

rdd_numeri = sc.parallelize([12, 3, 7, 8, 14, 21, 16, 9, 46, 11, 2])

pari_dispari = rdd_numeri.map(lambda n: ('DISPARI' if n%2 else 'PARI',n))
reduced_pari_dispari = pari_dispari.reduceByKey(lambda a,b:a+b)
ris = reduced_pari_dispari.collect()

print(ris)
sc.stop()
```

```
[('PARI', 98), ('DISPARI', 51)]
```

Lo scopo di *map* è quello di emettere, per ogni numero in input, una coppia chiave/valore composta da:

- **una stringa** - PARI o DISPARI a seconda che il numero sia divisibile o meno per due;
- **il numero stesso**.

L'operazione di *reduce* viene applicata mediante il metodo *reduceByKey*() che **raccoglie le coppie per chiave** esaminandone i valori con l'espressione:

lambda a, b: a+b

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

È molto importante comprendere il ruolo delle variabili *a* e *b*. A seguito del lavoro del *map* si avranno due gruppi:

1. **i numeri etichettati con PARI**;
2. **i numeri etichettati con DISPARI**.

Tali gruppi verranno elaborati separatamente e ogni loro valore passerà per la variabile *b*.

Secondo la regola *a+b*, ogni valore assunto da *b* sarà sommato con il risultato parziale contenuto in *a*. Al termine dell'elaborazione di ogni gruppo, la somma dei suoi valori sarà contenuta in *a.*

Una simile concatenazione di operazioni può non risultare subito immediata, ma è possibile avvicinarvisi gradualmente.

Per familiarizzare con essa, si può svolgere prima separatamente il *map* associandogli direttamente un *collect* come mostrato nel codice seguente.

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

rdd_numeri = sc.parallelize([12, 3, 7, 8, 14, 21, 16, 9, 46, 11, 2])

pari_dispari = rdd_numeri.map(lambda n: ('DISPARI' if n%2 else 'PARI',n))
ris = pari_dispari.collect()

print(ris)
sc.stop()
```

```
[('PARI', 12), ('DISPARI', 3), ('DISPARI', 7), ('PARI', 8), ('PARI', 14), ('DIS
PARI', 21), ('PARI', 16), ('DISPARI', 9), ('PARI', 46), ('DISPARI', 11), ('PAR
I', 2)]
```

Si avrà così evidenza della formazione delle coppie. Si vede chiaramente, infatti, che ogni valore è accompagnato dalla chiave che rappresenta il suo essere divisibile per 2.

L'algoritmo **MapReduce** permette di implementare una gran varietà di operazioni, ma Spark mette a disposizione metodi ancora più comodi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Maggi G. (2020). Data science con Python. Edizioni LSWR.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).